



TẬP ĐOÀN BƯU CHÍNH VIỄN THÔNG VIỆT NAM
VNPT IT - TRUNG TÂM DMS
Bộ phận Phát triển Giải pháp AI

BÁO CÁO ĐÁNH GIÁ PHƯƠNG ÁN CHUYỂN CƠ SỞ DỮ LIỆU SANG ORACLE CHO HỆ THỐNG TEXT-TO-SQL

Kiểm chứng đầu nối, chuyển đổi dữ liệu
và đo tải đối chứng Oracle 26ai Free với PostgreSQL Neon

PHẠM VI BÁO CÁO

Báo cáo trả lời câu hỏi “**có nên chuyển cơ sở dữ liệu của hệ thống Text-to-SQL từ PostgreSQL sang Oracle hay không**” bằng số liệu đo được, không bằng suy luận. Nội dung gồm bốn phần: dựng một instance Oracle thật và kiểm chứng tăng đầu nối; chuyển toàn bộ 94 bảng dữ liệu nghiệp vụ sang Oracle; cho ứng dụng chạy thật trên Oracle; và đo tải đối chứng ở mức 50 người dùng đồng thời giữa hai cơ sở dữ liệu trên cùng một máy, cùng một cấu hình suy luận.

Tóm tắt thông số

Cơ sở dữ liệu thử nghiệm:	Oracle AI Database 26ai Free 23.26.2.0.0, cài tại chỗ
Cơ sở dữ liệu đối chứng:	PostgreSQL trên Neon, region ap-southeast-1
Dữ liệu chuyển đổi:	94/94 bảng, 11.687/11.687 dòng, đối chiếu khớp tuyệt đối
Kiểm chứng đầu nối:	7/7 phép thử đạt
Đo tải:	6 lượt ở mức 50 người dùng, 100% trả về dữ liệu thật
Độ trễ tầng CSDL:	Oracle nhANH HƠN 6,4 LẦN (15 ms so với 94 ms)
Thông lượng hệ thống:	Oracle CÒN 82% (4,58 so với 5,60 req/giây)
Kết luận:	Không nên đổi vì lý do hiệu năng

Người thực hiện:

Lê Đức Anh

Trung tâm DMS

Ngày 10 tháng 08 năm 2026

Mục lục

1	Tóm tắt kết quả cho Lãnh đạo	2
2	Môi trường và phương pháp	3
3	Chuyển đổi dữ liệu	4
4	Kiểm chứng tăng đầu nối	5
5	Kết quả đo tải	6
6	Phương ngữ SQL	8
7	Kiểm thử Text-to-SQL trên truy vấn JOIN nhiều bảng	9
8	Các lỗi phát hiện trong quá trình thực hiện	11
9	Kết luận và khuyến nghị	12
	Phụ lục. Nguồn số liệu	13

1. Tóm tắt kết quả cho Lãnh đạo

1.1. Kết luận

Khuyến nghị

Nếu lý do là hiệu năng: không nên đổi. Cơ sở dữ liệu chiếm dưới 1% độ trễ của hệ thống. Đặt Oracle trên cùng máy chủ với tầng suy luận còn làm thông lượng giảm 18%, vì Oracle giành CPU và bộ nhớ với chính nút thắt của hệ thống.

Nếu lý do là yêu cầu nghiệp vụ: làm được. Toàn bộ phần kỹ thuật đã được kiểm chứng chạy thật trong báo cáo này. Chi phí còn lại nằm ở việc sửa phương ngữ SQL và kiểm thử lại chất lượng sinh SQL, không nằm ở hạ tầng.

1.2. Ba con số cần nhớ

Bảng 1: Tóm tắt kết quả đo tải ở mức 50 người dùng đồng thời

Chỉ số	Oracle tại chỗ	PostgreSQL Neon	Chênh lệch
Truy vấn chỉ chạm CSDL (trung vị)	15 ms	94 ms	Oracle nhanh hơn 6,4 lần
Thông lượng toàn hệ thống	4,58 req/s	5,60 req/s	Oracle còn 82%
Câu hỏi đi qua mô hình (trung vị)	55,3 giây	42,8 giây	Oracle chậm hơn 29%

Số liệu là trung bình các lượt đo đã ấm. Cả hai cột dùng cùng một máy, cùng cấu hình Ollama (OLLAMA_NUM_PARALLEL=4, GPU tích hợp tắt), cùng bộ câu hỏi, cùng kịch bản k6. Biến duy nhất thay đổi là cơ sở dữ liệu.

1.3. Vì sao nhanh hơn 6 lần mà vẫn lỗ

Hệ thống có hai nhánh xử lý tách biệt. Năm trong sáu câu hỏi của bộ kiểm thử được nhận dạng theo mẫu và sinh thẳng câu lệnh SQL, **không gọi mô hình ngôn ngữ**. Chỉ một câu đi qua tầng suy luận — và chính câu đó chiếm gần như toàn bộ thời gian.

Đổi cơ sở dữ liệu chỉ tác động vào nhánh thứ nhất, nhánh vốn đã nhanh. Rút 94 miligiây xuống 15 miligiây là cải thiện thật, nhưng nó nằm trong một request mất 42–55 giây. Trong khi đó, việc đặt thêm một tiến trình Oracle lên cùng máy lại lấy mất CPU và RAM của llama.cpp — tức là lấy đúng vào chỗ đang là nút thắt.

Nói gọn

Phương án này làm cho **dưới 1% nhanh lên 6 lần**, đổi lấy **99% còn lại chậm đi gần một phần ba**.

2. Môi trường và phương pháp

2.1. Cấu hình môi trường

Bảng 2: Môi trường kiểm thử

Thành phần	Cấu hình
Máy chủ thử nghiệm	Windows 11 Home, 13,8 GB RAM, APU Radeon 780M
Cơ sở dữ liệu thử nghiệm	Oracle AI Database 26ai Free 23.26.2.0.0, cài tại chỗ, PDB FREEPDB1, listener cổng 1521
Cơ sở dữ liệu đối chứng	PostgreSQL trên Neon, region ap-southeast-1 (Singapore)
Thư viện đầu nối	python-oracledb 4.0.2, chế độ thin (không cần Oracle Client)
Tầng suy luận	Ollama llama3.2 3B, OLLAMA_NUM_PARALLEL=4, GPU tích hợp tắt
Ứng dụng	Vanna 2.0 trên FastAPI/Uvicorn, 1 worker
Công cụ đo tải	k6, kịch bản ramping-vus, cao điểm 50 người dùng

2.2. Tiêu chí thành công

Một request chỉ được tính là thành công khi **có dữ liệu thật trả về**, tức luồng SSE phải chứa sự kiện `dataframe`. Tiêu chí “HTTP 200 kèm [DONE]” bị loại bỏ vì nó không phát hiện được câu trả lời rỗng: ứng dụng vẫn đóng luồng bình thường và vẫn kết bằng câu “Đã hoàn tất truy vấn” ngay cả khi tầng SQL hỏng hoàn toàn.

Điều này không phải giả định lý thuyết. Trong quá trình thực hiện báo cáo này, lỗi đó đã xảy ra thật và được mô tả ở Mục 8.

2.3. Cách bố trí phép so sánh

Điểm yếu thường gặp của loại so sánh này là thay đổi nhiều biến cùng lúc. Để tránh, các lượt đo được bố trí như sau:

- cả hai cơ sở dữ liệu đo trên **cùng một máy**, trong **cùng một phiên làm việc**, với Oracle chạy nền ở cả hai trường hợp;
- cấu hình Ollama giữ nguyên tuyệt đối giữa các lượt;
- biến duy nhất thay đổi là công tắc `DATABASE_BACKEND`;
- mỗi cấu hình đo **nhiều lượt**, vì độ dao động giữa các lượt lớn (xem Mục 5.2).

3. Chuyển đổi dữ liệu

3.1. Kết quả

Toàn bộ schema qlsp_backup được chuyển từ PostgreSQL sang Oracle bằng script loadtest/nap_du_lieu_oracle.py.

Bảng 3: Kết quả chuyển đổi dữ liệu

Hạng mục	Nguồn (PostgreSQL)	Đích (Oracle)
Số bảng	94	94
Số dòng	11.687	11.687
Đối chiếu từng bảng	—	Khớp toàn bộ

Script có thể chạy lại nhiều lần: mỗi lượt xoá schema cũ rồi dựng lại từ đầu, nên không tích lũy sai lệch. Sau khi chép xong, script tự đối chiếu số dòng hai bên từng bảng một và báo lệch nếu có.

3.2. Ánh xạ kiểu dữ liệu

Bảng 4: Ánh xạ kiểu dữ liệu PostgreSQL sang Oracle

PostgreSQL	Oracle	Ghi chú
text, character varying	VARCHAR2(n) / CLOB	Chọn theo độ dài thực tế; nhân 3 cho UTF-8
uuid	VARCHAR2(36)	
jsonb, json, ARRAY	VARCHAR2 / CLOB	Lưu nguyên văn JSON
boolean	NUMBER(1)	0/1, chạy được trên mọi bản Oracle
smallint, integer, bigint	NUMBER(5/10/19)	
numeric	NUMBER(p,s)	Giữ nguyên độ chính xác
timestamp without time zone	TIMESTAMP	
timestamp with time zone	TIMESTAMP WITH TIME ZONE	

Một điểm cần lưu ý khi vận hành: hai cột trong dữ liệu nghiệp vụ trùng từ khoá Oracle — product_lifecycle_history.comment và spdv_lifecycle_timeline.date. Script đã tự bọc dấu nháy kép cho hai cột này, nhưng **mọi câu SQL sinh ra sau này cũng phải bọc nháy**, nếu không sẽ lỗi cú pháp. Đây là loại lỗi chỉ xuất hiện khi có ai đó hỏi đúng vào hai bảng ấy.

4. Kiểm chứng tầng đầu nối

Trước khi đo tải, tầng đầu nối được kiểm chứng riêng bằng bộ kiểm thử `loadtest/kiem_chung_oracle.py`. Bộ này chỉ đọc, không tạo, sửa hay xóa đối tượng nào trong cơ sở dữ liệu.

Bảng 5: Kết quả 7 phép thử đầu nối

#	Phép thử	Số đo	Kết quả
1	Đầu nối và phiên bản	Oracle 26ai Free 23.26.2.0.0, chế độ thin	ĐẠT
2	Độ trễ khi pool đã ấm	Trung vị 0,5 ms; p95 0,8 ms	ĐẠT
3	Không chặn vòng lặp sự kiện	Trễ tối đa 6,4 ms trên 133 mẫu, trong khi có truy vấn ngủ 2 giây	ĐẠT
4	Truy vấn song song thật	8 truy vấn ngủ 2 giây xong trong 2,61 giây (hệ số 6,1×)	ĐẠT
5	Phương ngữ SQL	5/5 điểm đúng như dự đoán	ĐẠT
6	Câu lệnh không trả về bảng	Khởi PL/SQL trả về <code>rows_affected</code>	ĐẠT
7	Cắt truy vấn quá hạn	Cắt sau 1,09 giây khi đặt <code>call_timeout</code> 0,5 giây	ĐẠT

4.1. Độ trễ thuần của tầng cơ sở dữ liệu

Phép thử số 2 cho con số đáng chú ý nhất về lợi ích của việc đặt cơ sở dữ liệu tại chỗ:

Bảng 6: Độ trễ một truy vấn tối giản khi connection pool đã ấm

Cấu hình	Trung vị	p95
PostgreSQL trên Neon, Singapore	105 ms	—
Oracle tại chỗ (loopback)	0,5 ms	0,8 ms

Nhanh hơn khoảng **200 lần**, lớn hơn cả ước lượng ban đầu là “dưới 10 miligiây”. Cần đặt đúng tỉ lệ: đó là tiết kiệm 0,105 giây trong một request mất 45 giây — cải thiện 0,2%.

4.2. Kiểm chứng ngược phép thử số 3

Phép thử quan trọng nhất là phép thử số 3, vì nó phát hiện lỗi chặn vòng lặp sự kiện — loại lỗi từng làm tỉ lệ lỗi của hệ thống lên 13% ở đợt kiểm thử đầu tiên. Để chắc chắn nó không phải phép thử luôn báo đạt, phép thử được chạy ngược trên một hàm giả lập:

Bảng 7: Kiểm chứng ngược: bộ phát hiện có phân biệt được hai kiểu cài đặt không

Kiểu cài đặt	Độ trễ vòng lặp sự kiện	Kết luận
Gọi thẳng thư viện đồng bộ (bản cũ)	1.990 ms	KHÔNG ĐẠT
Đẩy sang thread pool (bản đã sửa)	6,6 ms	ĐẠT
Số đo thật trên Oracle	6,4 ms	ĐẠT

Ngưỡng 200 ms nằm gọn giữa hai giá trị, và số đo thật trên Oracle khớp với về “đã sửa”.

5. Kết quả đo tải

5.1. Toàn bộ các lượt đo

Bảng 8: Sáu lượt đo ở mức 50 người dùng đồng thời, 100% trả về dữ liệu thật

Lượt đo	Thông lượng	Truy vấn CSDL	Câu qua LLM	p95 câu LLM
Oracle, lượt 1 (<i>ngươi</i>)	2,94	16 ms	87,7 s	116,6 s
Oracle, lượt 2	4,13	14 ms	55,9 s	87,4 s
Oracle, lượt 3	5,02	15 ms	54,8 s	57,0 s
PostgreSQL, lượt 1 (10/08)	5,10	98 ms	44,0 s	70,8 s
PostgreSQL, lượt 2 (10/08)	6,10	91 ms	41,6 s	44,6 s
PostgreSQL, mốc gốc (05/08)	6,06	105 ms	45,3 s	47,8 s

Thông lượng tính bằng req/giây. “Truy vấn CSDL” là trung vị của năm câu hỏi không gọi mô hình. Mốc gốc 05/08 tái lập được trong hôm nay (6,10 so với 6,06), nên bản thân mốc chuẩn vẫn dùng được.

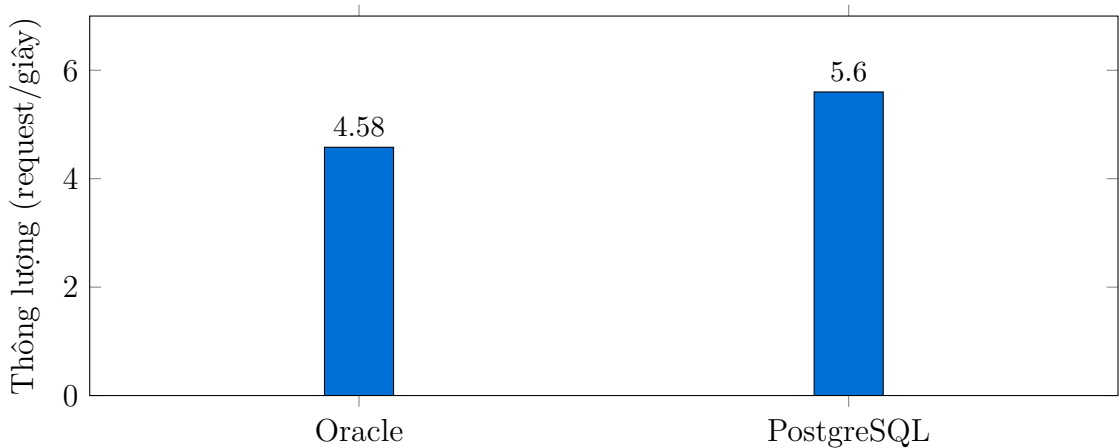
5.2. Vì sao phải đo nhiều lượt

Một đỉnh chính về phương pháp

Bản đầu của phân tích này kết luận “Oracle làm hệ thống chậm đi gấp đôi” dựa trên **đúng một lượt đo**. Lượt đó là lượt Oracle nguội, khi cache của cơ sở dữ liệu chưa ấm. Ba lượt Oracle liên tiếp cho 2,94 → 4,13 → 5,02 req/giây — tức **độ dao động giữa các lượt còn lớn hơn cả hiệu ứng đang muốn đo**. Kết luận cũ là nói quá và đã được sửa bằng số liệu trong báo cáo này.

Bài học vận hành: khi tự đo lại, phải chạy ít nhất ba lượt rồi bỏ lượt đầu. Một lượt duy nhất trên Oracle nguội sẽ cho kết luận sai lệch tới gần hai lần.

5.3. So sánh các lượt đã âm



Hình 1: Thông lượng trung bình các lượt đã âm: Oracle đạt 82% của PostgreSQL

Bảng 9: So sánh trực tiếp, chỉ tính các lượt đã âm

Chỉ số	Oracle tại chỗ	PostgreSQL Neon	Chênh lệch
Thông lượng	4,58 req/s	5,60 req/s	Oracle còn 82%
Câu qua mô hình (trung vị)	55,3 giây	42,8 giây	Oracle chậm hơn 29%
Truy vấn chỉ chạm CSDL	15 ms	94 ms	Oracle nhanh hơn 6,4 lần
Tỉ lệ trả về dữ liệu thật	100%	100%	Không chênh

5.4. Cô lập nguyên nhân

Để xác định nguyên nhân của phần lỗi, một lượt đo riêng được thực hiện với ứng dụng vẫn dùng PostgreSQL, nhưng Oracle chạy nền mà không phục vụ gì. Kết quả: thông lượng đã giảm và câu qua mô hình đã chậm thêm, dù ứng dụng không gửi một truy vấn nào tới Oracle.

Điều đó loại trừ giả thuyết “Oracle xử lý truy vấn chậm” và xác nhận nguyên nhân thật: Oracle giành CPU và bộ nhớ với llama.cpp. Trên máy 13,8 GB RAM đang phải nạp một mô hình 3,8 GB, việc thêm một tiến trình cơ sở dữ liệu chiếm khoảng 600 MB thường trú là đủ để làm chậm nhánh suy luận.

6. Phương ngữ SQL

Mã nguồn và chỉ thị cho mô hình đang gắn chặt với PostgreSQL. Bảng dưới đây đã được kiểm chứng từng dòng trên Oracle thật.

Bảng 10: Các điểm khác biệt phương ngữ, đã kiểm chứng bằng số đo

Cú pháp PostgreSQL	Kết quả trên Oracle	Cách thay
<code>LIMIT 10</code>	ORA-03049	<code>FETCH FIRST 10 ROWS ONLY</code>
<code>information_schema.columns</code>	ORA-00942	<code>ALL_TAB_COLUMNS</code>
<code>FROM bang AS u</code>	ORA-03048	<code>FROM bang u (bỏ AS)</code>
<code>search_path</code>	Không có	<code>ALTER SESSION SET CURRENT_SCHEMA</code>
<code>SELECT</code> không <code>FROM</code>	Chạy được	Chỉ từ Oracle 23ai trở đi
<code>AS</code> trước bí danh cột	Chạy được	Giữ nguyên

Hai dòng ORA-03048 và `SELECT` không `FROM` **không có trong bản phân tích ban đầu** — chúng chỉ lộ ra khi chạy thật. Đây là minh họa cụ thể cho việc phân tích tĩnh không thay thế được đo đạc.

7. Kiểm thử Text-to-SQL trên truy vấn JOIN nhiều bảng

Điều bộ kiểm thử tải KHÔNG chứng minh được

Sáu câu hỏi của bộ kiểm thử tải đều là **COUNT(*)** trên **một bảng**. Chúng chứng minh **đầu nối được**, chứ không chứng minh **sinh SQL đúng**. Mục này bổ sung phần còn thiếu đó.

Bộ kiểm thử `loadtest/thu_join_nhieu_bang.py` gồm 8 câu hỏi cần JOIN 2–3 bảng. Mỗi câu có một đáp án đúng do chính script tính bằng SQL viết tay, sau đó mới hỏi ứng dụng và so hai bên. Một câu chạy được nhưng ra số sai vẫn tính là hỏng.

Bảng 11: Kết quả trên truy vấn JOIN nhiều bảng, cùng bộ câu hỏi cho hai CSDL

Tiêu chí	Oracle	PostgreSQL
Có bảng trả về (tiêu chí cũ)	7/8	8/8
Trả lời đúng câu được hỏi	1/8	1/8

Hai cơ sở dữ liệu cho kết quả giống hệt nhau, nên đây không phải lỗi của Oracle. Đây là khiếm khuyết sẵn có của ứng dụng, chỉ là chưa ai đo nên chưa lộ ra.

7.1. Ba nguyên nhân

1. **Bộ khớp mẫu cướp đường câu hỏi — 5/8 câu.** Hàm `_known_counts_tool_call` trong `main.py` thấy chữ “có bao nhiêu” là sinh thẳng một câu đếm đóng sẵn gồm nhiều bảng không liên quan, **không hề gọi mô hình**. Nhận ra được vì các câu này trả lời trong 0,0–0,1 giây. Người dùng hỏi “bao nhiêu đơn vị *không có* người dùng nào” (đáp án đúng: 2) thì nhận về tổng số người dùng là 2.580.
2. **Mô hình 3B sinh SQL sai khi phải JOIN — 1/8 câu.** Câu duy nhất thoát được bộ khớp mẫu và cần GROUP BY thì sinh ra `SELECT u.full_name, u.unit_id, COUNT(*) ... GROUP BY u.unit_id`, Oracle trả ORA-00979.
3. **Chỉ 1/8 câu thật sự đúng:** một JOIN hai bảng kèm SUM, ra đúng 1.174.446.607.589.

7.2. Một cái bẫy dữ liệu

Cột `plan_items.doanh_thu_du_kien` nghe như số tiền nhưng là VARCHAR2 chứa văn bản mô tả (“Tiết kiệm 20% thời gian xử lý”). Bất kỳ ai — người hay mô hình — thử SUM lên cột đó đều nhận ORA-01722. Cột số thật là `estimated_cost`.

7.3. Hệ quả cho phương án chuyển đổi

Việc phải làm dù ở lại hay chuyển đi

Phần rủi ro về chất lượng sinh SQL không những có thật mà còn lớn hơn dự đoán, và nó **không liên quan tới việc đổi cơ sở dữ liệu**. Sửa nó là việc riêng, phải làm dù ở lại PostgreSQL hay chuyển sang Oracle: thu hẹp bộ khớp mẫu để nó không nuốt các câu hỏi phức tạp, và dùng mô hình mạnh hơn cho nhánh sinh SQL.

8. Các lỗi phát hiện trong quá trình thực hiện

Phần này ghi lại bốn lỗi phát hiện được nhờ chạy thật. Cả bốn đều đã sửa.

8.1. Mô hình phớt lờ chỉ thị phương ngữ

Ở lượt chạy thử đầu tiên trên Oracle, năm câu khớp mẫu chạy tốt nhưng câu duy nhất đi qua mô hình thì hỏng: llama3.2 3B sinh ra ... ORDER BY id LIMIT 5, Oracle trả ORA-03049. Chỉ thị Oracle lúc đó nằm ở mục 9, cuối prompt hệ thống.

Chuyển nguyên khối chỉ thị lên **ngay đầu prompt**, kèm ví dụ SAI/ĐÚNG cụ thể, thì mô hình tuân thủ và cả 6/6 câu đều chạy. Đây đúng là rủi ro đã được cảnh báo từ đầu, nhưng nó sửa được bằng cách viết lại chỉ thị, **không cần đổi mô hình**.

8.2. Ứng dụng báo thành công khi truy vấn thất bại

Lỗi nghiêm trọng nhất về mặt vận hành. Khi `run_sql` thất bại, ứng dụng vẫn đóng luồng SSE bình thường và vẫn trả lời:

“Đã hoàn tất truy vấn. Kết quả chính xác được hiển thị trong bảng phía trên.”

mặc dù không có bảng nào. Nghĩa là một tầng SQL hỏng hoàn toàn trông giống hết một tầng khoẻ mạnh — với người dùng, với log, và với bất kỳ phép đo tải nào chỉ kiểm tra luồng có kết thúc hay không.

Đã sửa: ghi log kèm câu SQL mỗi khi truy vấn thất bại, và trả về thông điệp nói đúng sự thật kèm mã lỗi từ cơ sở dữ liệu.

8.3. Lỗi cắt dấu chấm phẩy của khối PL/SQL

Lớp `OracleRunner` cắt dấu chấm phẩy cuối câu lệnh — đúng với SQL thường, nhưng sai với khối PL/SQL, vì `END;` là cú pháp bắt buộc. Mọi khối PL/SQL đều nhận PLS-00103. Lỗi này có sẵn trong mã nguồn gốc và chỉ lộ ra khi dấu nổi thật.

8.4. Oracle không tự phục hồi sau khi khởi động lại máy

Sau một lần khởi động lại, ứng dụng chết hoàn toàn với DPY-6001: `Service ``freepdb1'' is not registered with the listener`. Chẩn đoán cho thấy instance **vẫn OPEN** và PDB **vẫn READ WRITE** — chỉ là instance không đăng ký được với listener.

Nguyên nhân: hai file `listener.ora` và `tnsnames.ora` do bộ cài sinh ra dùng tên máy `AnhLD.lan`, mà tên đó không phân giải được khi máy đổi mạng. Đã sửa bằng cách trỏ `local_listener` thẳng vào `127.0.0.1` và thay tên trong cả hai file.

Điểm cần đưa vào phương án vận hành

Một cơ sở dữ liệu đặt tại chỗ trên máy thường xuyên đổi mạng cần cấu hình listener theo **địa chỉ tuyệt đối**. Nếu không, mỗi lần khởi động lại là một lần hệ thống chết, và triệu chứng lại trông giống như cơ sở dữ liệu hỏng chứ không phải lỗi cấu hình mạng.

9. Kết luận và khuyến nghị

9.1. Trả lời câu hỏi đặt ra

1. **Đổi sang Oracle vì hiệu năng: không nên.** Cơ sở dữ liệu chiếm dưới 1% độ trễ. Ngay cả khi giả sử Oracle nhanh vô hạn, p95 của hệ thống chỉ giảm dưới 1%. Thực tế đo được còn tệ hơn giả thuyết: thông lượng giảm 18% vì Oracle giành tài nguyên với tầng suy luận.
2. **Đổi sang Oracle vì nghiệp vụ: hạ tầng làm được.** 7/7 phép thử đầu nổi đạt, 94/94 bảng chuyển đúng, và bộ kiểm thử tải chạy 100% request trả về dữ liệu thật — nhưng bộ đó chỉ gồm câu COUNT(*) một bảng.
3. **Nút thắt thật nằm ở chất lượng sinh SQL, và nó có sẵn từ trước.** Trên truy vấn JOIN nhiều bảng, ứng dụng chỉ trả lời đúng 1/8 câu — và con số này **giống hệt nhau trên cả Oracle lẫn PostgreSQL**, nên không phải do đổi cơ sở dữ liệu. Chi tiết ở Mục 7. Đây là việc phải sửa dù ở lại hay chuyển đi.

9.2. Giới hạn của kết luận

Điều báo cáo này KHÔNG kết luận

Số liệu đo trên **một máy duy nhất** vừa chạy mô hình vừa chạy cơ sở dữ liệu. Báo cáo bác bỏ phương án **cài Oracle lên chính máy chủ ứng dụng**, chứ **không** bác bỏ phương án đặt Oracle trên một **máy chủ riêng** trong mạng nội bộ — khi đó vẫn giữ được lợi ích độ trễ mà không phải chia sẻ CPU với tầng suy luận. Nếu bên nghiệp vụ có sẵn máy chủ Oracle riêng, độ trễ kỳ vọng nằm giữa hai cột: tốt hơn 105 ms của Neon, kém hơn 0,5 ms của loopback.

9.3. Việc nên làm tiếp để giảm độ trễ

Ngân sách công sức nên dồn vào tầng suy luận, nơi chiếm trên 99% độ trễ:

1. dò lại OLLAMA_NUM_PARALLEL sau khi đã bật GPU tích hợp, vì ràng buộc bộ nhớ đã khác — khoảng 20 phút;
2. xác định ngưỡng an toàn của GPU tích hợp trong khoảng 30–50 người dùng — khoảng 10 phút;
3. đầu tư GPU rời hoặc chuyển sang API LLM đám mây, nếu cần đạt p95 dưới 10 giây ở mức 30–50 người dùng. Đây là con đường duy nhất còn lại, vì mọi phương án cấu hình không tốn chi phí đều đã thử hết.

Phụ lục. Nguồn số liệu

Mọi con số trong báo cáo này đều tái lập được từ các file sau, có trong kho mã nguồn:

File	Nội dung
loadtest/nap_du_lieu_oracle.py	Script chuyển dữ liệu, tự đổi chiều số dòng
loadtest/kiem_chung_oracle.py	Bộ 7 phép thử đầu nối, tự sinh báo cáo
loadtest/bao_cao_dau_noi_oracle.md	Báo cáo đầu nối do máy sinh
loadtest/k6_50_oracle_np4_v2.json	Đo tải Oracle, lượt 1 (ngủ)
loadtest/k6_50_oracle_np4_lan2_v2.json	Đo tải Oracle, lượt 2
loadtest/k6_50_oracle_np4_lan3_v2.json	Đo tải Oracle, lượt 3
loadtest/k6_50_pg_hnay_lan1_v2.json	Đo tải PostgreSQL, lượt 1
loadtest/k6_50_pg_hnay_lan2_v2.json	Đo tải PostgreSQL, lượt 2
loadtest/k6_50_np4_v2.json	Đo tải PostgreSQL, mốc gốc 05/08
loadtest/k6_50_pg_kem_oracle_v2.json	Lượt cô lập nguyên nhân
phan_tich_doi_sang_oracle.md	Phân tích chi tiết kèm diễn giải

Cách chạy lại bộ kiểm chứng đầu nối:

```
set ORACLE_USER=qlsp_backup
set ORACLE_PASSWORD=...
set ORACLE_DSN=localhost:1521/freepdb1
python loadtest/kiem_chung_oracle.py
```

Cách chuyển ứng dụng giữa hai cơ sở dữ liệu: đặt biến DATABASE_BACKEND bằng oracle hoặc postgres trong file .env.